

Coursework 2 (summative)

● Graded

Student
Simon Lefort

Total Points
46 / 50 pts

Question 1

1. Claim 1 7 / 9 pts

+ 9 pts Correct

✓ + 7 pts Right idea. Minor error in proof.

+ 5 pts Right idea. Proof lacks a part.

+ 3 pts Some vague idea, but not a proof.

+ 0 pts Incorrect or missing.

💬 You did well generally. The definitions of accepting/rejecting states were swapped relative to the requirements of the prompt

Question 2

1. Claim 2. 9 / 9 pts

✓ + 9 pts Correct

+ 7 pts Right idea. Minor error in proof.

+ 5 pts Right idea. Proof lacks a part.

+ 3 pts Some vague idea, but not a proof.

+ 0 pts Incorrect or missing.

Question 3

2(a) 7 / 7 pts

✓ + 7 pts Correct

+ 6 pts Mostly correct. Minor error.

+ 5 pts Mostly correct. Small errors.

+ 3 pts Right idea but serious errors.

+ 0 pts Incorrect or missing.

Question 4

2(b) 2 / 4 pts

✓ + 2 pts Reduction correct.

+ 1 pt Reduction only partly.

+ 0 pts Reduction wrong or missing.

+ 2 pts Type correct.

+ 0 pts Type incorrect or missing.

💬 tao is the result type and you provided a type judgement but did not state the most general type.

Question 5

2(c) 4 / 4 pts

✓ + 2 pts Reduction correct.

+ 1 pt Reduction only partly.

+ 0 pts Reduction wrong or missing.

✓ + 2 pts Type correct.

+ 0 pts Type incorrect or missing.

Question 6

3(a) 3 / 3 pts

✓ + 3 pts Correct

+ 2 pts Mostly OK with minor error.

+ 0 pts Incorrect or missing.

Question 7

3(b) 2 / 2 pts

✓ + 2 pts Correct

+ 0 pts Incorrect or missing.

Question 8

3(c) 3 / 3 pts

✓ + 3 pts Correct

+ 2 pts Mostly OK with minor error.

+ 0 pts Incorrect or missing.

💬 The overall semantic meaning is clear and correct, the notation is more like data structure rather than standard set-theory notation though.

Question 9

3(d) 2 / 2 pts

✓ + 2 pts Correct

+ 0 pts Incorrect or missing.

💬 Instead of checking all vertices, we should say for all edges; as your mathematical notation in the end of the sentence shows.

✓ + 7 pts Correct

+ 6 pts Almost correct with very minor error/omission.

+ 5 pts Mostly correct.

+ 3 pts Some right ideas, but not a correct reduction.

+ 0 pts Wrong or missing.

Coursework 2 ITCS

Problem 1

a)

Because $\bar{A}$ and $\bar{B}$ are Turing-recognizable, we can construct :

- a machine that accepts $\bar{A}$, $M_{\bar{A}}$
- a machine that accepts $\bar{B}$, $M_{\bar{B}}$

Now, we want to create a machine that allows to separate $\bar{A}$ and $\bar{B}$. We build a machine M_C that uses interleaving and, as long as the machine has not accepted :

- we make $M_{\bar{A}}$ advance by one step. if $M_{\bar{A}}$ accepts, then M accepts.
- we make $M_{\bar{B}}$ advance by one step. if $M_{\bar{B}}$ accepts, then M rejects.

Since $A \cap B = \emptyset$ then $\bar{A} \cup \bar{B} = \Sigma^*$ and each string will be accepted either by $M_{\bar{A}}$ or by $M_{\bar{B}}$ (we pick the earliest acceptance event, thus for no input will M_C loop forever, and M_C is decidable). (1)

We can thus create a language C :

- if $w \in A$, then $w \in \bar{B}$ because $A \cap B = \emptyset$, therefore M_C rejects, thus $A \subseteq \bar{C}$
- if $w \in B$, then $w \in \bar{A}$, therefore M_C rejects, thus $B \subseteq C$.
- otherwise, either M_C accepts or rejects but will necessarily halt (argument (1)) and is decidable.

Therefore, we have built a decidable language C s.t. $A \subseteq \bar{C}$ and $B \subseteq C$. $\square$

b)

We can build:

- $A = \{\langle M \rangle \mid M \text{ accepts } \langle M \rangle\}$.
- $B = \{\langle M \rangle \mid M \text{ rejects } \langle M \rangle\}$.

where $\langle M \rangle$ is the encoding of a Turing machine M . These two sets represents the machines that accept themselves, and those who reject themselves. We know that $A \cap B$ are therefore distinct, and turing recognizable (there is a machine that can recognize them but not necessarily decide on them, because some machines may not halt when being run on their own encoding).

Let's assume that there is a decidable language C such that:

- $A \subseteq C$
- $B \subseteq \overline{C}$

Therefore, C would be able to decide and separate properly on A and B , and there exists a machine M_C that always halt that decide on C .

We can now use diagonalization to come up with a contradiction, and build a machine D that on input M , run M_C on M to determine if it is in C or not.

If M_C accepts M , then D rejects it and if M_C rejects M , then D accepts it. D does the opposite of C .

Let's now run $\langle D \rangle$ on D itself and assume that:

- $\langle D \rangle \in C$: M_C accepts $\langle D \rangle$ and therefore D rejects $\langle D \rangle$ so $\langle D \rangle \in B$ but $B \subseteq \overline{C}$ so $\langle D \rangle \in C$ cannot be true.
- $\langle D \rangle \notin C$: M_C rejects $\langle D \rangle$ and therefore D accepts $\langle D \rangle$ so $\langle D \rangle \in A$ but $A \subseteq C$ so $\langle D \rangle \notin C$ cannot be true.

We end up in a contradiction showing us that C cannot exist.

Problem 2

a)

Let us recall the abstraction rule :

$$\frac{x : \sigma, \Gamma \vdash t : \tau}{\Gamma \vdash (\lambda x : \sigma. t) : \sigma \rightarrow \tau} (\rightarrow_I)$$

and the elimination rule :

$$\frac{\Gamma \vdash t : \sigma \rightarrow \tau \quad \Gamma \vdash u : \sigma}{\Gamma \vdash (tu) : \tau} (\rightarrow_E)$$

We are starting with the following context $\Gamma = \{x : \text{nat}, f : \text{nat} \rightarrow \text{nat}\}$.

$$\frac{\Gamma \vdash f : (\text{nat} \rightarrow \text{nat}) \quad \Gamma \vdash x : \text{nat}}{\Gamma \vdash fx : \text{nat}} (\rightarrow_E)$$

$$\frac{\Gamma \vdash fx : \text{nat} \quad \Gamma \vdash f : (\text{nat} \rightarrow \text{nat})}{\Gamma \vdash f(fx) : \text{nat}} (\rightarrow_E)$$

$$\frac{x : \text{nat}, \Gamma \vdash f(fx) : \text{nat}}{\Gamma \vdash (\lambda x. f(fx)) : (\text{nat} \rightarrow \text{nat})} (\rightarrow_I)$$

$$\frac{f : (\text{nat} \rightarrow \text{nat}), \Gamma \vdash (\lambda x : \text{nat}. f(fx)) : (\text{nat} \rightarrow \text{nat})}{\Gamma \vdash (\lambda f. \lambda x. f(fx)) : ((\text{nat} \rightarrow \text{nat}) \rightarrow (\text{nat} \rightarrow \text{nat}))} (\rightarrow_I)$$

We proved that the final expression has type $(\text{nat} \rightarrow \text{nat}) \rightarrow (\text{nat} \rightarrow \text{nat})$.

b)

We are given the following expression:

$$(\lambda x. \lambda y. xy)yx$$

Without renaming, substitution of x by y would mistakenly capture the inner y . We have to α -rename the inner bound name y to some unused variable, such as b :

$$(\lambda x. \lambda b. xb)yx$$

This can be justified because both are α -equivalent:

$$\lambda y. xy \equiv_{\alpha} \lambda b. xb$$

Let's apply the first β -reduction $((\lambda x. t)u \rightarrow_{\beta} t \left[\frac{u}{x} \right])$:

$$(\lambda x. \lambda b. xb)yx \rightarrow_{\beta} (\lambda b. yb)x$$

and the second:

$$(\lambda b. yb)x \rightarrow_{\beta} yx$$

Let's α -rename x to a so it makes our explanations clearer:

$$(\lambda a. \lambda b. ab)yx$$

This can be justified because they are α -equivalent:

$$(\lambda x. \lambda b. xb)yx \equiv_{\alpha} (\lambda a. \lambda b. ab)yx$$

Let's assume that first, $\Gamma_1 = \{a : \sigma, b : \tau\}$. Because in this expression $(\lambda a. \lambda b. ab)yx$ we end up with a result of type ab , we know that a is a function that takes the type of b (τ) as a parameter and outputs some value of unknown type ν , and therefore a has to be of type $\tau \rightarrow \nu$.

$\Gamma_2 = \{a : \tau \rightarrow \nu, y : \tau \rightarrow \nu, b : \tau\}$. Because y is given as an argument for a , it has to be of type a .

Our expression is $(\lambda a. \lambda b. ab)yx$:

$$\frac{b : \tau, \Gamma_2 \vdash a : \tau \rightarrow \nu}{\Gamma_2 \vdash (\lambda b. ab) : (\tau \rightarrow \nu)} (\rightarrow_I)$$

Because x is given as parameter for this function, it must be of type τ .

$\Gamma_3 = \{a : \tau \rightarrow \nu, y : \tau \rightarrow \nu, b : \tau, x : \tau\}$.

$$\frac{\frac{a : \tau \rightarrow \nu, \Gamma_3 \vdash (\lambda b. ab) : (\tau \rightarrow \nu)}{\Gamma_3 \vdash (\lambda a. \lambda b. ab) : (\tau \rightarrow \nu) \rightarrow (\tau \rightarrow \nu)} (\rightarrow_I)}{\Gamma_3 \vdash ((\lambda a. \lambda b. ab)y) : (\tau \rightarrow \nu)} (\rightarrow_E)$$

$$\frac{\Gamma_3 \vdash (\lambda a. \lambda b. ab)y : (\tau \rightarrow \nu) \quad \Gamma_3 \vdash x : \nu}{\Gamma_3 \vdash ((\lambda a. \lambda b. ab)yx) : \tau} (\rightarrow_E)$$

Therefore, the final type of the term is ν .

c)

$$P = (\lambda x. \lambda y. y(xy))(\lambda x \lambda y. y(xy))y$$

Let's α -rename:

$$(\lambda x. \lambda a. a(xxa))(\lambda x \lambda y. y(xy))y$$

First step:

$$(\lambda x. \lambda a. a(xxa))(\lambda x \lambda y. y(xy))y \rightarrow_\beta (\lambda a. a((\lambda x \lambda y. y(xy))(\lambda x \lambda y. y(xy))a))y$$

Second step:

$$(\lambda a. a((\lambda x \lambda y. y(xy))(\lambda x \lambda y. y(xy))a))y \rightarrow_\beta y((\lambda x \lambda y. y(xy))(\lambda x \lambda y. y(xy))y) = y(P)$$

We know that if a term is simply typed, it can always be reduced to a normal form in a finite number of steps (strong normalization property). However, as we have seen while reducing this lambda terms, we obtain an infinite recursion. Therefore, it cannot be well-typed.

Problem 3

a)

Let's formulate the h – SCHEDULE problem precisely.

An instance D of this problem consists in:

- a set of exams $X = \{x_1, \dots, x_m\}$
- a set of students $S = \{s_1, \dots, s_n\}$
- an integer $h \in \mathbb{N}$
- a function $\text{ExamStudents} : X \rightarrow \mathcal{P}(S)$ where $\mathcal{P}(S)$ is the powerset of S , mapping each exam to the set of students taking it.

To check if an instance is part of Q_1 , we must verify whether there exists a function $\text{Slot} : X \rightarrow [[1, h]]$ such that:

$$(\forall x, y \in X, (\text{ExamStudents}(x) \cap \text{ExamStudents}(y) \neq \emptyset) \wedge x \neq y \rightarrow \text{Slot}(x) \neq \text{Slot}(y)) \wedge (\forall x \in X, 1 \leq \text{Slot}(x) \leq h)$$

b)

Let's assume that we are given a function $\text{Slot} : X \rightarrow [[1, h]]$.

In a first place, we check that the provided slot function is providing us with correct values:

$$\forall x \in X, 1 \leq S(x) \leq h \text{ in } \mathcal{O}(m).$$

Then, we check that, for all pairs of exams (x, y) :

- either, we have $\text{Slot}(x) \neq \text{Slot}(y)$
- or for all students s , we have $s \notin \text{ExamStudents}(x) \vee s \notin \text{ExamStudents}(y)$

This corresponds to verifying the constraint defined at Question 1 and can be checked $\mathcal{O}(m^2n)$.

c)

Let's formulate the k – COLOURING problem precisely.

An instance D of this problem consists in:

- $k \in \mathbb{N}$, the number of colours
- $G = \langle V, E \rangle$, an undirected graph where V is the set of the vertices and E is the set of the edges. We are defining each edge as having a property left and a property right, giving us without loss of generality access to both vertices connected by this edge.

Let's introduce a new function $\text{Colour} : V \rightarrow [[1, k]]$ such that $\text{Colour}(x)$ represents the color of the vertex x .

Then, we must verify whether it exists a function $\text{Colour} : V \rightarrow [[1, k]]$ such that:

$$\forall e \in E, \text{Colour}(e.\text{left}) \neq \text{Colour}(e.\text{right})$$

d)

Let's assume that we are given a function $\text{Colour}(v) : V \rightarrow [[1, k]]$.

Then, we need to check that for all vertices, the color on the left is different than the color on the right. Therefore, this can be verified in $\mathcal{O}(E)$.

e)

We need to find a Turing transducer such that, for any instance $d = (h, X, S, \text{ExamStudents}) \in D_1$, it outputs an instance $f(d) = (k, G) \in D_2$ of the K-Colour problem. Moreover, we want $d \in Q_1$ (the scheduling instance is satisfiable) if and only if $f(d) \in Q_2$ (the corresponding colouring instance is colourable).

We can notice that by representing exams as vertices and colours as slots, we can connect two vertices with an edge whenever the corresponding exams share at least one student, since such exams must not receive the same colour (and share the same slots).

Therefore, let's define $f : D_1 \rightarrow D_2$ by:

$$f((h, X, S, \text{ExamStudents})) = (k, G = (V, E))$$

where $k = h$, $V = X$, and $E = \{(x_i, x_j) \mid \text{ExamStudents}(x_i) \cap \text{ExamStudents}(x_j) \neq \emptyset\}$.

This is computable in polynomial time (in $\mathcal{O}(m^2n)$) by checking all the pairs of exams in $\mathcal{O}(m^2)$ then by computing the difference of the sets in $\mathcal{O}(n)$.

To prove that it is correct, we need to:

1. prove that if there exists a valid schedule, then the corresponding instance of the colouring problem would be colorable. We can assume that we have an existing valid schedule :

$$\text{Slot} : X \rightarrow [[1, h]]$$

such that no student takes two exams in the same time slot.

We can construct a colouring for G by setting $\text{Colour}(x) = \text{Slot}(x)$. For every edge $(x_i, x_j) \in E$, the exams share a student (by definition of E , this ensures that $\text{Colour}(x_i) \neq \text{Colour}(x_j)$). Therefore, we found a proper colouring.

2. if there exists a valid k -colouring, with $k = h$:

$$\text{Colour} : V \rightarrow [[1, k]]$$

such that for every edge $(x_i, x_j) \in E$, $\text{Colour}(x_i) \neq \text{Colour}(x_j)$. We can construct a scheduling by setting $\text{Slot}(x) = \text{Colour}(x)$. Since any pair of exams sharing a student are connected by an edge, and edges connect only differently coloured vertices, no student has two exams in the same slot. Therefore, this is a proper h -schedule.